

A Three-Tool Build Journal

Moodboard Studio • eBay Trend Finder • Trend Finder Upgrade

A self-learning write-up: how each tool was built, the architecture and data flow, what worked, what broke (and how it was fixed), and the transferable lessons for future projects.

Project: Front-end design studio + web-scraping/AI research tools
Stack: HTML/CSS/JS • Node.js (Express) • Python (Flask) • BeautifulSoup • feedparser • Gemini Flash • Kie AI
Generated: 15 June 2026

The three tools at a glance

Tool	What it does	Backend	Ports
Moodboard Studio	Turns one brief (idea/theme/products) into 6 homepage design moodboards as AI images	Node + Kie AI	3000 / —
Trend Finder (eBay)	Search a topic → top 5 trending products (name, image, store, buy link) + dated CSV / printable report	Flask + BeautifulSoup	3000 / 5000
Trend Finder Upgrade	Search trending articles → Gemini extracts products + ideas (labeled) → you pick → dated CSV	Flask + Gemini, Node proxy	3002 / 5001

1 — Moodboard Studio

The original tool. A single-page app that takes a creative brief and generates six homepage design directions (two Minimalist, two Contemporary, two Dynamic), each rendered as an image via Kie AI.

How it was built

- **Front end** (`Frontend.html`): one self-contained file — inline CSS design system (warm paper palette, Fraunces/Newsreader/JetBrains Mono fonts) and inline JS. State persists in `localStorage` so edits survive a refresh.
- **Backend** (`server.js`, Express): serves the page and *proxies* Kie AI so the API key stays server-side. Flow: POST a prompt → get a `taskId` → poll until the image is ready.
- **Prompt templates** (`concepts.js`): the six concepts are templates with `{idea}/{theme}/{products}` placeholders — editable without touching app logic.

Key pattern: the async create-then-poll proxy

Image generation is slow, so the API is split into two calls. The browser never sees the key; the server does the waiting.

```
POST /api/generate/image → { taskId }
GET /api/task/image?taskId=... → { state: pending|success|fail, urls[] }
```

What worked

- A single inline-everything HTML file is fast to iterate on and trivial to share.
- Proxying the third-party API through your own server is the clean way to hide keys and dodge browser CORS.
- Separating prompt templates from logic made the creative part editable in isolation.

What to watch

- Generated image URLs live on the provider's CDN and expire — export to keep a copy.
- Polling needs a timeout/back-off so a stuck task doesn't spin forever.

2 — Trend Finder (eBay product scraper)

The first Python tool. A search box returns the top 5 trending products for any topic — name, image, store/source, and a button to the real listing — scraped live from eBay. No fake data, no API key.

How it was built

- **New page** (`trend-finder.html`) reusing the Moodboard design system, linked from a new nav bar added to both pages.
- **Flask API** (`app.py`, port 5000): GET `/api/trends?q=` requests eBay's search page and parses it with BeautifulSoup.
- The frontend fetches the API across origins (CORS enabled) and renders result cards.

The real lesson: scraping is a moving target

This is where most of the learning happened. Three concrete problems showed up — each is a transferable lesson:

Problem	Symptom	Fix
---------	---------	-----

Bot blocking	Bare request → HTTP 403	Send a real browser User-Agent; warm up cookies by hitting the homepage first
Markup drift	Old selectors (. s - item) found nothing	eBay had moved to . s - card ; inspect the live DOM, don't trust assumptions
Intermittent challenge	'Pardon Our Interruption' page ~30% of calls	Detect the challenge page and retry 2-3x with a short delay

Defensive parsing throughout: skip the 'Shop on eBay' placeholder, treat the static placeholder graphic as 'no image', de-dupe by item id, and return an empty list (never fabricated data) when nothing parses.

What worked

- Isolating all selectors in one function (`search_ebay()`) made the inevitable markup fix a one-place change.
- The retry-on-challenge loop turned a flaky ~70% success rate into reliable results.
- Returning a clean empty state kept the honesty rule intact: no sample data, ever.

Dated reports (added later)

- Every result set carries a server-stamped capture date (captured in the API), shown as 'Captured YYYY-MM-DD' — because trends change, a report is only reproducible if it says *when* it was taken.
- Two one-click outputs: a date-stamped **CSV** (`ebay-trends-<topic>-<date>.csv`) and a **printable report** window (Save-as-PDF) — both client-side, no new backend.

What didn't

- First selector set was based on memory of eBay's old HTML — wrong. Always verify against the live page.
- A single request with no UA/cookies never works against a big retailer.

3 — Trend Finder Upgrade (article → idea research)

The most ambitious tool. Search trending *articles* across three sources, use Gemini Flash to read each article and extract its meaningful content — the **products it recommends AND the article's main ideas**, each labeled by type — let the user pick which to keep, and export to a dated CSV. Gemini is used for text analysis only — never images.

Architecture (two servers, by design)

Server	Port	Responsibilities
Flask server.py	5001	All scraping + AI: /api/trending (Google News RSS + Reddit + Hacker News) and /api/extract (fetch → clean text → Gemini → fallback). Holds the Gemini key.
Node serve.mjs	3002	Serves the page and proxies thumbnail images (/img?url=) to dodge hotlink/CORS blocks.

Data flow

```
search → GET :5001/api/trending?q= → 5 article cards (+ captured date)
card button → GET :5001/api/extract?url= → { items:[{rank,text,type}], method, captured }
items → Idea Panel (checkboxes) → Preview table → Export dated CSV (client-side Blob + BOM)
```

Smart bits

- **AI-first extraction, heuristic fallback:** Gemini decides what the article's real content is; if the key is missing or the call fails, a BeautifulSoup heuristic (/ or repeated headings) keeps the tool usable.
- **Products AND ideas, labeled by type:** the prompt asks Gemini for both the products/tools the article recommends and its main ideas/takeaways, each tagged product or idea — so it works on essays, not just listicles (the earlier version only scraped raw lists).
- **Structured AI output:** Gemini is asked for JSON (responseMimeType: application/json) so results parse reliably instead of scraping prose.
- **User selects, not auto-pick:** **Extract Ideas** adds the whole list in one click; **Browse & Pick** opens a checklist that starts empty so the user ticks exactly what to keep before adding.
- **Dated, reproducible export:** a server-stamped capture date flows into the panel, Preview, and a date-stamped trend-ideas-<date>.csv with a Type column.
- **Google News workaround:** News links are browser-only redirects, so the server flags needs_manual and the UI shows a paste box — open the article, copy the real URL, paste it back.

Problems hit & fixed (live debugging)

Problem	Root cause	Fix
Reddit returned 403	.json endpoint now blocks bots	Switched to Reddit's RSS feed + retry on 429
Only News results showed	HN is tech-focused; 0 hits for consumer queries	Expected — interleave 3 sources, degrade gracefully
Image proxy '400/404'	Not a bug — passing through upstream status for bad test URLs	Confirmed with known-good images (200)
Gemini 429 on 1st call	Free tier had limit: 0 for gemini-2.0-flash	Probed models; switched default to gemini-2.5-flash

Biggest lesson here

Read the actual error body. The Gemini failure looked like ordinary rate limiting, but the JSON said limit: 0 for that specific model — a quota/availability issue, not a throttle. Probing each model against the key found a working one in seconds.

What this mini-project teaches

Architecture & integration

- **Proxy third-party APIs through your own backend.** It hides keys, fixes CORS, and lets you reshape responses. Used for Kie, Gemini, and images.
- **Polyglot is fine.** Node and Python servers coexisted happily; pick the right tool per job (Node for the existing site + image streaming, Python for scraping/AI) and let CORS bridge them.
- **Run tools as separate services on separate ports.** Each tool stayed independent, so a change in one couldn't break another — lowest-risk way to add features.
- **Design once, reuse everywhere.** A small set of CSS tokens (colors, fonts, card/button styles) copied across pages gave a consistent look with no framework.

Working with live web data

- **The web actively resists scraping.** Expect 403/429/captcha. Browser headers, cookie warm-ups, and retry loops are the baseline, not extras.
- **Markup changes — isolate your selectors.** Keep parsing in one function so the inevitable fix is contained.
- **Prefer feeds/APIs over HTML when they exist.** RSS (Google News, Reddit) and JSON APIs (HN Algolia) are far more stable than scraping pages.
- **Degrade gracefully and never fake data.** Empty states and honest errors beat fabricated results.

Working with AI (LLMs)

- **Let the model do judgement, not plumbing.** Gemini decided which text was 'the list' — far better than brittle HTML rules — while code handled fetching and formatting.
- **Ask for structured output.** JSON mode + a clear schema makes results parseable and reliable.
- **Always keep a non-AI fallback.** Keys expire, quotas hit zero, calls fail — the tool should still do something useful.
- **Model availability is not uniform.** Free-tier quota differs per model; make the model configurable and verify before assuming.

Debugging mindset

- **Reproduce in isolation.** Small standalone scripts (one request, print the status/body) found every root cause faster than guessing in the app.
- **Read the full error, not just the status code.** The 429 'limit: 0' detail and the eBay challenge-page title were the whole story.
- **Verify end-to-end with the real thing.** curl against live endpoints caught the eBay markup change, the Reddit block, and the Gemini quota — none were visible from the code alone.

Suggested next steps for self-study

- Add caching (e.g. 10-minute TTL) to scrape/AI calls to cut repeat latency and respect rate limits.
- Add a tiny test that hits each live source and warns when a selector/feed breaks.
- Try swapping the eBay scraper for an official API to compare reliability vs. effort.

- Add request back-off + a queue so bursts of extractions stay under the free-tier limit automatically.

Built iteratively, debugged against the live web and a live LLM. The recurring theme: external systems are unreliable — design for it, observe carefully, and always leave an honest fallback.